

Reviewer Pack — Plethysm Coefficient Exponential Gap in Symmetric Squares of...

Entry 2515221a101f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 22:01:41 UTC

Plethysm Coefficient Exponential Gap in Symmetric Squares of Permutation Representations for Random CNFs

Entry ID: 2515221a101f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 22:01:41 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality, Plethysm, Algebraic Combinatorics **Field B** (complexity object): ACC⁰ Circuit Size

Statement:

For a random CNF formula with n variables, the plethysm coefficient of the symmetric square of the permutation representation decomposed into irreducibles is $\Omega(2^n)$, which implies that ACC⁰ circuits cannot compute the permanent of such formulas.

Rationale (proposer's reasoning):

The exponential growth of plethysm coefficients for symmetric squares of permutation representations suggests deep structural differences between permanent and determinant, potentially exposing lower bounds on ACC⁰ circuit size.

Taxonomy category: GCT_DET_PERM (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7ec83c317baa7e88

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.8s

5.1 Generated Python source

```
import random
import math

def factorial(n):
    if n == 0:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def hook_length(young_diagram):
    m, n = len(young_diagram), len(young_diagram[0])
    hook_lengths = [[0] * n for _ in range(m)]

    for i in range(m - 1, -1, -1):
        for j in range(n - 1, -1, -1):
            hook_lengths[i][j] = young_diagram[i][j] + (m - i) + (n - j) - 1

    return hook_lengths

def plethysm_coefficient(young_diagram):
    m, n = len(young_diagram), len(young_diagram[0])
    hook_lengths = hook_length(young_diagram)

    numerator = factorial(m + n)
    denominator = 1
    for i in range(m):
        for j in range(n):
            denominator *= hook_lengths[i][j]

    return numerator / denominator

def generate_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = [random.randint(1, n) * (2 * random.choice([0, 1]) - 1) for _ in range(random.randint(1, n))]
        cnf.append(clause)
    return cnf
```

```

def cnf_to_young_diagram(cnf, n):
    young_diagram = [[0] * n for _ in range(n)]

    for clause in cnf:
        for literal in clause:
            abs_literal = abs(literal)
            if abs_literal <= n:
                row = abs(abs_literal - 1) // (n + 1 - abs(abs_literal))
                col = abs(abs_literal - 1) % (n + 1 - abs(abs_literal))
                young_diagram[row][col] += 1

    return young_diagram

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5): # Aim for at least 30 instances per seed
            cnf = generate_cnf(n, random.randint(1, n * (n - 1) // 2))
            young_diagram = cnf_to_young_diagram(cnf, n)
            plethysm_coeff = plethysm_coefficient(young_diagram)

            total_metric_value += plethysm_coeff
            instances_tested += 1

    mean_metric_value = total_metric_value / instances_tested
    conjecture_holds = mean_metric_value >= 2 ** n_values[-1]

    return {
        "metric_name": "plethysm_coefficient",
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):

```

```

    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

tested": 30, "conjecture_holds": False, "counterexample": ""}
TRIAL: {"seed": 593, "metric_name": "plethysm_coefficient", "metric_value": 5.052894465957788e-
12, "instances_tested": 30, "conjecture_holds": False, "counterexample": ""}
TRIAL: {"seed": 631, "metric_name": "plethysm_coefficient", "metric_value": 8.018485485069505e-
12, "instances_tested": 30, "conjecture_holds": False, "counterexample": ""}
TRIAL: {"seed": 677, "metric_name": "plethysm_coefficient", "metric_value": 7.608903062601938e-
12, "instances_tested": 30, "conjecture_holds": False, "counterexample": ""}
TRIAL: {"seed": 727, "metric_name": "plethysm_coefficient", "metric_value": 3.770662207648113e-
12, "instances_tested": 30, "conjecture_holds": False, "counterexample": ""}
TRIAL: {"seed": 773, "metric_name": "plethysm_coefficient", "metric_value": 7.053303052209947e-
12, "instances_tested": 30, "conjecture_holds": False, "counterexample": ""}
TRIAL: {"seed": 821, "metric_name": "plethysm_coefficient", "metric_value": 5.589756678349182e-
12, "instances_tested": 30, "conjecture_holds": False, "counterexample": ""}
TRIAL: {"seed": 877, "metric_name": "plethysm_coefficient", "metric_value": 6.7381142507277544e-
12, "instances_tested": 30, "conjecture_holds": False, "counterexample": ""}
TRIAL: {"seed": 929, "metric_name": "plethysm_coefficient", "metric_value": 2.9968433257691583e-
12, "instances_tested": 30, "conjecture_holds": False, "counterexample": ""}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_92e1a0ef.py", line 111, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_92e1a0ef.py", line 111, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	59048
2	preregistration	ollama_remote	qwen3:8b	0	0	24470
3	novelty	ollama_remote	qwen3:8b	0	0	20747
4	novelty	ollama_remote	qwen3:8b	0	0	17009
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13136
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11491
7	judge	ollama_remote	qwen3:8b	0	0	23356

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 169256 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/2515221a101f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2515221a101f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2515221a101f.tar.gz` (if generated)